

Esercitazione 10: Last Problem Session on DP

Giacomo Paesani

May 20, 2026

Esercizio 1. Fornire in pseudo-codice un algoritmo che data una sequenza finita di numeri interi X restituisce la lunghezza della più lunga sotto-sequenza alternata Y . Quindi se ad esempio abbiamo che la sequenza X è data da $(1, 3, 8, 5, 4, 2, 6, 0, 1, 2, 8, 9, 5)$ allora si ottiene $Y = (3, 8, 2, 6, 0, 9, 5)$. Implementare questo algoritmo in modo che il tempo di esecuzione sia al più $\mathcal{O}(n^2)$ (ma si può fare anche in $\mathcal{O}(n)$). Come deve essere modificato l'algoritmo per far sì che restituisca una sotto-sequenza strettamente crescente di lunghezza massima?

Soluzione 1. Questo problema può essere risolto in tempo $\mathcal{O}(n)$. Infatti è sufficiente scorrere il vettore X e salvare delle informazioni nelle variabili *inc* e *dec*: *inc* indica la lunghezza della più lunga sotto-sequenza alternata di X_i al passo i -esimo dell'algoritmo tale che l'ultimo valore di questa sequenza è maggiore del precedente, *dec* indica la lunghezza della più lunga sotto-sequenza alternata di X_i al passo i -esimo dell'algoritmo tale che l'ultimo valore di questa sequenza è minore del precedente. Come devo però aggiornare questi valori? Il valore di *inc* deve essere aggiornato se e solo se l'ultimo elemento della sequenza alternata era più piccolo dell'elemento precedente, cioè se sto partendo dalla sotto-sequenza rappresentata da *dec*, e valore di *dec* deve essere aggiornato se e solo se l'ultimo elemento della sequenza alternata era più grande dell'elemento precedente, cioè se sto partendo da la sotto-sequenza rappresentata da *inc*. Lo pseudo-codice è rappresentato nel Algoritmo 1. $\square$

Algorithm 1

Input: vettore X di lunghezza n

Output: lunghezza della più lunga sotto-sequenza alternata di X .

1: **function** **AlternateSubSequence**(X, n)

2: $inc = 1$

3: $dec = 1$

4: $op = nil$

5: $inc_last = X[1]$

6: $dec_last = X[1]$

7: $i = 2$

8: **while** $i \leq n$ **do**

9: **if** $X[i] > dec_last$ **then**

10: **if** $op \neq inc$ **then**

11: $inc = dec + 1$

12: $op = inc$

13: $inc_last = X[i]$

14: **if** $dec = 1$ **then**

15: $dec_last = X[i]$

16: **if** $X[i] < inc_last$ **then**

17: **if** $op \neq dec$ **then**

18: $dec = inc + 1$

19: $op = dec$

20: $dec_last = X[i]$

21: **if** $inc = 1$ **then**

22: $inc_last = X[i]$

23: $i++$

24: **return** $\max\{inc, dec\}$

	i=2	i=3	i=4	i=5	i=6	i=7	i=8	i=9	i=10	i=11	i=12	i=13
inc=1	2	2	2	2	2	4	4	6	6	6	6	6
dec=1	1	3	3	3	3	5	5	5	5	5	5	7
op=nil	inc	inc	dec	dec	dec	inc	dec	inc	inc	inc	inc	dec
inc_last=1	3	8	8	8	8	6	6	1	2	8	9	9
dec_last=1	3	8	5	4	2	2	0	0	0	0	0	5

Esercizio 2. Si considera una griglia $n \times n$ con $n > 0$. Un cammino su questa griglia deve partire dalla cella di coordinate $(0, 0)$ in alto a sinistra e

deve arrivare alla posizione di coordinate $(n - 1, n - 1)$ in basso a destra. E' possibile muoversi solo su celle adiacenti, andando di un passo verso il basso o di un passo verso destra. Inoltre, sono vietati i cammini che toccano le celle di coordinate (i, j) con $i > j$, cioè non è permesso andare sotto la diagonale che va da $(0, 0)$ a $(n - 1, n - 1)$. Fornire in pseudo-codice un algoritmo che calcoli il numero di cammini validi con un tempo di esecuzione $\mathcal{O}(n^2)$.

Soluzione 2. Si costruisce una matrice M di dimensioni $n \times n$ tale che alla fine dell'algoritmo, il valore di $M[i, j]$ indica il numero di cammini validi dalla cella di coordinate $(0, 0)$ alla cella di coordinate (i, j) , sfruttando la ricorsione. Iniziamo con i casi base; se la cella di coordinate (i, j) si trova

- sulla prima riga, cioè $i = 0$, allora $M[i, j] = 1$ perché la si può raggiungere solo facendo passi verso destra e quindi in un unico modo;
- sulla diagonale e non è la cella di partenza, cioè $0 < i = j$, allora $M[i, j] = M[i - 1, j]$ perché la si può solo raggiungere solo dalla casella consecutiva in alto;
- sotto la diagonale, cioè se $i > j$, allora $M[i, j] = 0$ perché non è possibile raggiungere tale cella con cammini validi.

Supponiamo infine che la cella in esame non soddisfa nessuna delle condizioni precedenti, allora $M[i, j] = M[i - 1, j] + M[i, j - 1]$ perché è possibile raggiungere questa cella sia da quella consecutiva in alto che da quella consecutiva a sinistra.

Algorithm 2 Algoritmo per calcolare il numero di cammini validi sopra la diagonale.

Input: intero n

Output: numero di cammini tra le due caselle

```

1: function PathsAboveDiagonal( $n$ )
2:   for  $i = 0, \dots, n - 1$  do
3:     for  $j = i, \dots, n - 1$  do
4:       if  $i = 0$  then
5:          $M[i, j] = 1$ 
6:       else if  $i = j$  then
7:          $M[i, j] = M[i - 1, j]$ 
8:       else
9:          $M[i, j] = M[i - 1, j] + M[i, j - 1]$ 
10:  return  $M[n - 1, n - 1]$ 

```

Un esempio del codice richiesto è dato dall'Algoritmo 2. La soluzione si conclude facendo una rapida analisi del costo computazionale: in pratica, l'algoritmo proposto consiste in due cicli **for**, uno contenuto nell'altro, in cui le operazioni interne si svolgono in $\mathcal{O}(1)$ e quindi complessivamente abbiamo $\mathcal{O}(n^2)$. $\square$

Esercizio 3. Dato un intero n , sia c_n il numero di stringhe binarie lunghe n in cui non compaiono due zeri consecutivi. Fornire uno pseudo-codice che descrive un algoritmo, che dato $n \geq 1$, calcola il valore di c_n in tempo $\mathcal{O}(n)$. E se invece volessi calcolare d_n , cioè il numero di stringhe binarie lunghe n in cui non compaiono tre zeri consecutivi?

Soluzione 3. Una delle possibili soluzioni è esposta nell'Algoritmo 3. L'idea è di costruire il caso base c_1 e poi ricorsivamente c_{n+1} a partire da c_n . Per semplificare la spiegazione di questo esercizio definiamo come a_n e b_n come il numero di stringhe binarie lunghe n in cui non compaiono due zeri consecutivi in cui l'ultimo elemento è uno 0 e un 1, rispettivamente. E' quindi chiaro che $c_n = a_n + b_n$.

Algorithm 3

Input: intero n

Output: numero di stringhe binarie di lunghezza n senza due 0 consecutivi

```

1: function NOCONSECUTIVEZEROS( $n$ )
2:    $i = 1$ 
3:    $a = 1$ 
4:    $b = 1$ 
5:    $c = 1$ 
6:   while  $i < n$  do
7:      $b = a$ 
8:      $a = c$ 
9:      $c = a + b$ 
10:     $i = i + 1$ 
```

Allora, una stringa rappresentata da a_{n+1} , e che quindi termina con 1, ha il prefisso lungo n in cui l'ultimo termine può essere sia uno 0 che un 1: abbiamo che $a_{n+1} = a_n + b_n = c_n$. Una stringa rappresentata da b_{n+1} , e che quindi termina con 0, ha il prefisso lungo n in cui l'ultimo termine deve essere necessariamente uno 1: abbiamo che $b_{n+1} = a_n$. Si osserva facilmente che

vengono calcolati esattamente n valori c_i e ognuno di questi calcoli si svolge in tempo $\mathcal{O}(1)$: l'algoritmo si risolve in $\mathcal{O}(n)$.

Per implementare la soluzione prima descritta è necessario occupare uno spazio pari a $\mathcal{O}(n)$ di memoria, cioè il valori a_i , b_i e c_i , per ogni $1 \leq i \leq n$. In realtà molti di questi dati diventano superflui una volta utilizzati: allora la variabile a contiene il valore di a_i , b contiene il valore di b_i e c contiene il valore di c_i e quindi per calcolare a_{i+1} , b_{i+1} e c_{i+1} è sufficiente aggiornare come prima descritto di valore di a , b e c e quindi usando $\mathcal{O}(1)$ spazio.

Supponiamo vogliamo calcolare d_n , allora definiamo a_n , b_n e c_n come il numero di stringhe di lunghezza n che terminano con 1, 10 e con 00, rispettivamente. Allora abbiamo necessariamente che $d_n = a_n + b_n + c_n$ e che $a_{n+1} = d_n$, $b_{n+1} = a_n$ e $c_{n+1} = b_n$. con il caso base: $d_1 = 2$, $a_2 = 2$, $b_2 = 1$ e $c_2 = 1$ e $d_2 = 4$. $\square$

Esercizio 4. Dati due interi k e n , con $1 \leq k \leq n$, definiamo $P(k, n)$ come il numero di differenti partizioni dei numeri da 1 a n in k sotto-insiemi non vuoti. Fornire in pseudo-codice un algoritmo che calcola $P(k, n)$ in tempo $\mathcal{O}(k \cdot n)$.

Soluzione 4. Per risolvere questo esercizio si suggerisce di adottare un approccio di programmazione dinamica. Costruiamo una matrice P con k righe ed n colonne. Come caso base, è facile osservare che $P(1, j) = 1$: se è possibile avere un solo sotto-insieme nella partizione, allora ogni numero deve essere necessariamente assegnato nell'unico sotto-insieme a disposizione.

Si considera una qualsiasi partizione dei numeri da 1 a j in i sotto-insiemi non vuoti e abbiamo due casi: o (1) il numero j è in un singleton della partizione oppure (2) il numero j non è in un singleton della partizione. Allora, il numero delle partizioni che soddisfano il caso (1) è pari alle partizioni dei numeri da 1 a $j - 1$ in $i - 1$ sotto-insiemi non vuoti e quindi $P(i - 1, j - 1)$. Infine, il numero delle partizioni che soddisfano il caso (2) è pari alle partizioni dei numeri da 1 a $j - 1$ in i sotto-insiemi non vuoti moltiplicato i cioè il numero di diversi elementi della partizione in cui il numero j può essere posizionato, e quindi $i \cdot P(i, j - 1)$. Allora in totale abbiamo che per $2 \leq i \leq j \leq n$ si ha che $P(i, j) = P(i - 1, j - 1) + i \cdot P(i, j - 1)$.

Algorithm 4

Input: interi k e n

Output: numero di partizioni in k sotto-insiemi non vuoti dei numeri da 1 a n

```
1: function COUNTPARTITION( $k, n$ )
2:   for  $i = 1, \dots, k$  do
3:     for  $j = 1, \dots, n$  do
4:       if  $i > j$  then
5:          $P(i, j) = 0$ 
6:       else if  $i = 1$  then
7:          $P(i, j) = 1$ 
8:       else
9:          $P(i, j) = P(i - 1, j - 1) + i \cdot P(i, j - 1)$ 
10:  return  $P(k, n)$ 
```

La soluzione proposta nell'Algoritmo 4 implementa proprio l'idea sopra descritta. Si osserva che il tempo di esecuzione dell'algoritmo è di $\mathcal{O}(k \cdot n)$: infatti ogni iterazione dei cicli **for** la computazione del numero $P(i, j)$ si esegue in tempo costante, cioè $\mathcal{O}(1)$. $\square$